



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

CPU 架构编程实验

颜国庄

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 4 月 3 日

目录

一、 问题一：矩阵与向量内积	1
(一) 实验介绍	1
(二) 算法设计	1
(三) 算法分析	1
二、 问题二:n 个数求和	3
(一) 问题介绍	3
(二) 算法设计	4
(三) 算法分析	4
三、 反思总结	6
(一) 收获	6
(二) 遇到的问题	6
四、 代码链接	6

一、 问题一：矩阵与向量内积

(一) 实验介绍

给定一个 $n \times n$ 矩阵，计算每一列与给定向量的内积，考虑两种算法设计思路：逐列访问元素的平凡算法和 cache 优化算法，进行实验对比：

1. 对两种思路的算法编程实现；
2. 练习使用高精度计时测试程序执行时间，比较平凡算法和优化算法的性能。

(二) 算法设计

我们要算一个 $n \times n$ 的矩阵和一个 n 维向量的内积，按常规直觉，肯定是一列一列地去遍历计算，这就是最普通的解法。但这种解法在实际运行中会遇到计算机底层存储机制的限制。因为在内存中，二维数组一般是按行连续存储的，而且 CPU 从内存读数据时，并不是用到哪个读哪个，而是一次性把一整块连续的数据读到 Cache 里。如果按普通算法逐列去遍历，指针在内存里的跳跃幅度就会非常大。一旦数据量变大，一行数据没法全部塞进一个缓存行里，Cache 的优势就完全发挥不出来了。这会导致极其频繁的缓存未命中，程序不得不频繁地向主存请求数据，极大地拖慢了计算时间。为了解决这个问题，我们引入了 Cache 优化算法。既然数据是按行存的，那我们就按行去遍历。具体来说，就是遍历矩阵的每一行时，把这一行里的每个元素分别跟向量里对应位置的元素相乘，然后再把结果累加起来。这种做法最大的好处就是顺应了数据的物理存储方式，一次内存访问就能把后续计算要用到的好几个数据都一起带进 Cache 里，命中率大幅提升，整体的数据读取和运算速度变快许多。

(三) 算法分析

对实验数据进行处理并画出对应的折线图后，我们可以直观地看出运行时间随问题规模增长的变化趋势。由数据和折线图可以看出，起初在问题规模 n 比较小的时候，三种算法的运行时间都极短，没有明显的差别。值得注意的是，在这个小规模区间出现了一些明显的数据异常波动，例如当 $n=60$ 时，平凡算法的运行时间出现了一个反常的峰值 (0.055 ms)。这可能是由于在数据规模极小时，核心计算的绝对耗时极短，此时程序的测量时间极易受到操作系统线程调度等系统背景噪音的干扰，从而掩盖了算法本身的真实效率。当问题规模提高之后，算法之间开始出现差距，优化算法的效率开始稳定高于平凡算法，但总体趋势大致相同；而当问题规模进一步扩大后，算法之间开始出现极其明显的差距，优化算法的效率显著高于平凡算法，cache 优化的效果极其显著。随着问题规模越来越大，数据在缓存中一次访存读取一整个缓存行的底层物理逻辑在处理算法时就占据了更主要的因素。与平凡算法跨缓存行的列式访问相比，行式访问时由于读取的数据在内存分布上是连续的，极大地提高了缓存命中率，故算法效率大大提高。对于 unroll 算法来说，其原理是把原来一路的运算，转换成了多路的计算。由最终的大规模数据可以看到，其优化效益也很显著，不仅同样规避了访存瓶颈，其运行效率甚至在绝大多数测试点微优于单纯的 cache 优化算法。

表 1: 平凡与优化算法运行时间比较

n	normal	cache	unroll	n	normal	cache	unroll	n	normal	cache	unroll
10	0.0034	0.0008	0.0005	100	0.0181	0.021	0.0169	1000	3.8166	1.7091	1.6532
20	0.0018	0.0015	0.0008	200	0.0716	0.0645	0.0645	2000	19.6122	7.0258	7.2582
30	0.0017	0.0028	0.0017	300	0.171	0.143	0.1437	3000	46.1731	15.0693	15.6793
40	0.0028	0.0038	0.0032	400	0.2992	0.2581	0.2518	4000	68.9985	24.4302	24.476
50	0.0041	0.0052	0.005	500	0.4805	0.4886	0.4272	5000	111.608	37.3174	37.6153
60	0.055	0.0069	0.0059	600	0.7184	0.5726	0.5779	6000	166.932	59.7896	57.9139
70	0.0168	0.0261	0.0171	700	0.9734	0.7656	0.7625	7000	225.439	72.7834	69.8154
80	0.0103	0.0116	0.011	800	1.3464	0.9981	1.0112	8000	311.637	97.7319	91.4089
90	0.0131	0.0293	0.0144	900	3.9161	1.3767	1.3884	9000	410.7	118.399	115.448

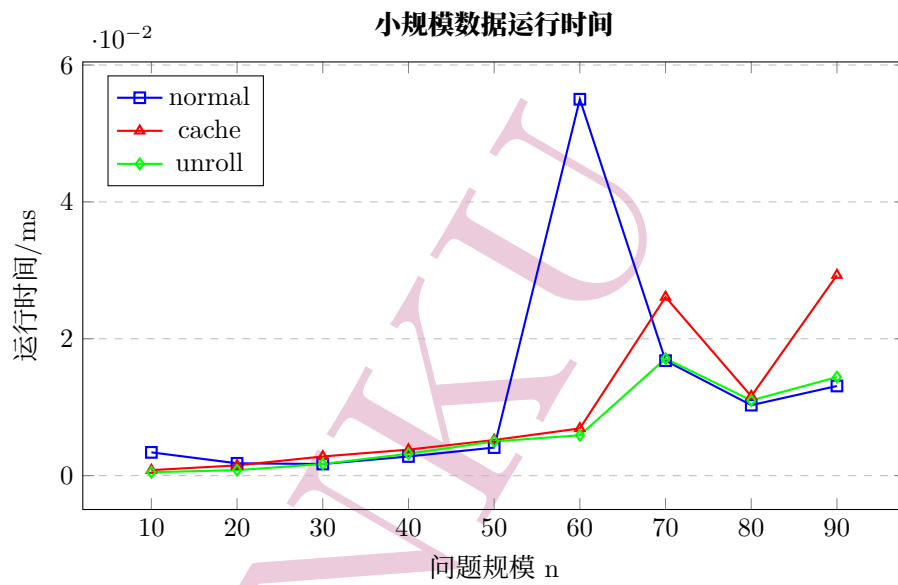


图 1: 小规模下矩阵与向量内积算法运行时间

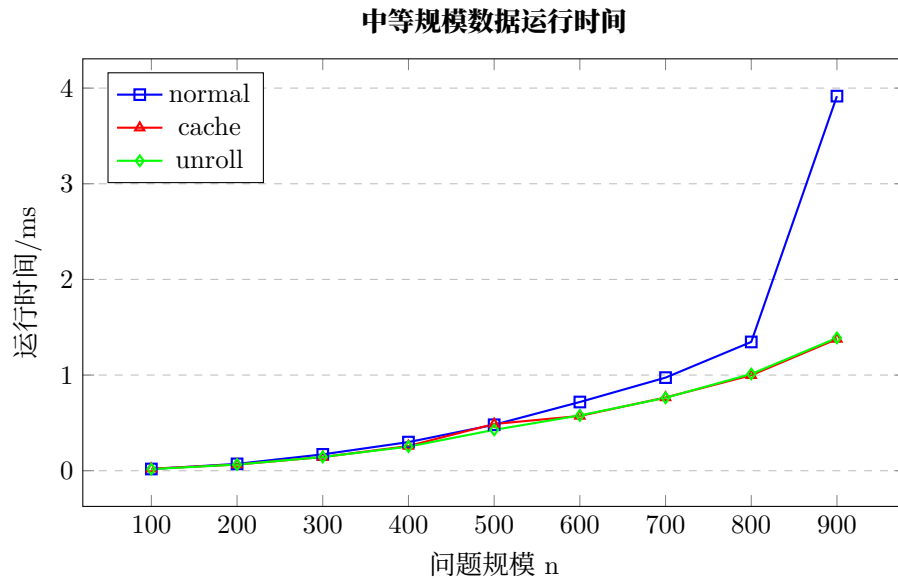


图 2: 中等规模下矩阵与向量内积算法运行时间

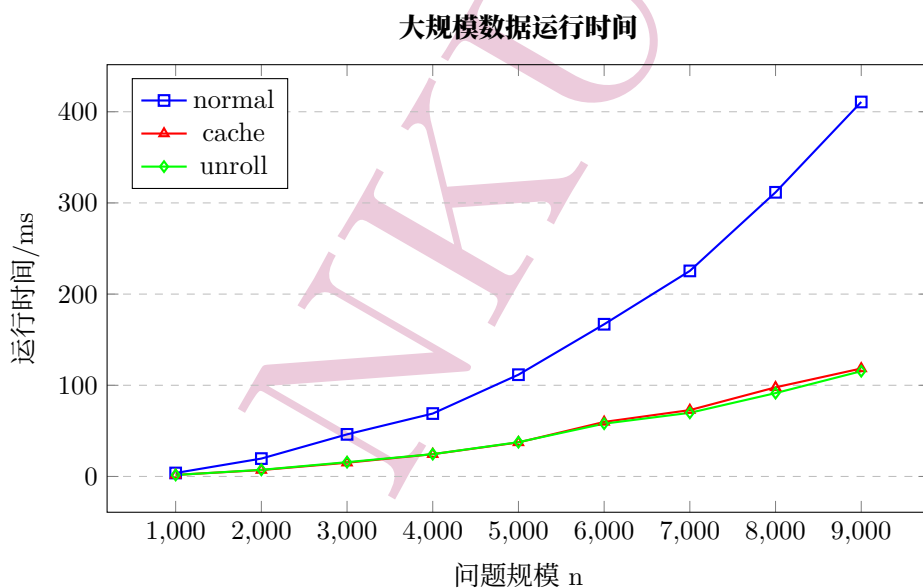


图 3: 大规模下矩阵与向量内积算法运行时间

二、 问题二:n 个数求和

(一) 问题介绍

计算 n 个数的和，考虑两种算法设计思路：逐个累加的平凡算法（链式）；适合超标量架构的指令级并行算法（相邻指令无依赖），如最简单的两路链式累加，再如递归算法——两两相加、中间结果再两两相加，依次类推，直至只剩下最终结果。完成如下作业：

1. 对两种算法思路编程实现；
2. 练习使用高精度计时测试程序执行时间，比较平凡算法和优化算法的性能。

(二) 算法设计

要实现 n 个数的求和。首先想到的就是直接逐个遍历 n 个元素，实现 n 个数的连续相加。这是平凡算法，但这种算法在底层执行时，后面的相加操作严格依赖于前面计算的结果，形成了严重的数据依赖。

由于现代计算机 CPU 普遍采用超标量架构，能够同时发射并运行多条互不依赖的指令，我们可以分多条线路求部分和，然后再把部分和加起来。基于这个原理，我们在代码中设计了两路链式累加。通过设置 `sum1` 和 `sum2` 两个独立的累加器分别处理相邻的元素。由于这两路是完全并行的，相邻的指令没有依赖关系，假如说一步相加操作需要一个时钟周期，那么就使得原来 n 个时钟周期完成的任务，我们只需 $n/2$ 个周期就可以完成了，显著提高了运行效率。

为了进一步探究优化的极限，我们在代码中又追加实现了四路循环展开。通过引入四个独立的累加器同时处理连续的四个元素，理论上可以将核心计算的耗时进一步压缩至约 $n/4$ 个周期。当问题规模 n 变得更大时，这种优化效果会更加明显。同时由于数据是 2 的 n 次幂，不需要对剩余尾部元素进行额外处理。经过分析，单纯递归算法的时间复杂度应为 $O(n \log n)$ ，我们在此不作数据分析。

(三) 算法分析

表 2: 平凡与多路展开优化算法运行时间比较 (单位: ms)

n	normal	multi	unroll	n	normal	multi	unroll
8	0.0002	0.0002	0.0002	16	0.0603	0.0442	0.0327
9	0.0005	0.0004	0.0003	17	0.2292	0.0881	0.0656
10	0.001	0.0016	0.0006	18	0.2395	0.177	0.261
11	0.0018	0.0015	0.0011	19	0.4786	0.6764	0.3817
12	0.0037	0.003	0.0023	20	0.9641	0.7061	0.5384
13	0.0075	0.0059	0.0041	21	1.9525	1.4574	1.0483
14	0.0149	0.0112	0.0082	22	3.8954	2.9685	2.2217
15	0.0298	0.029	0.0163	23	7.6989	6.1385	4.6767

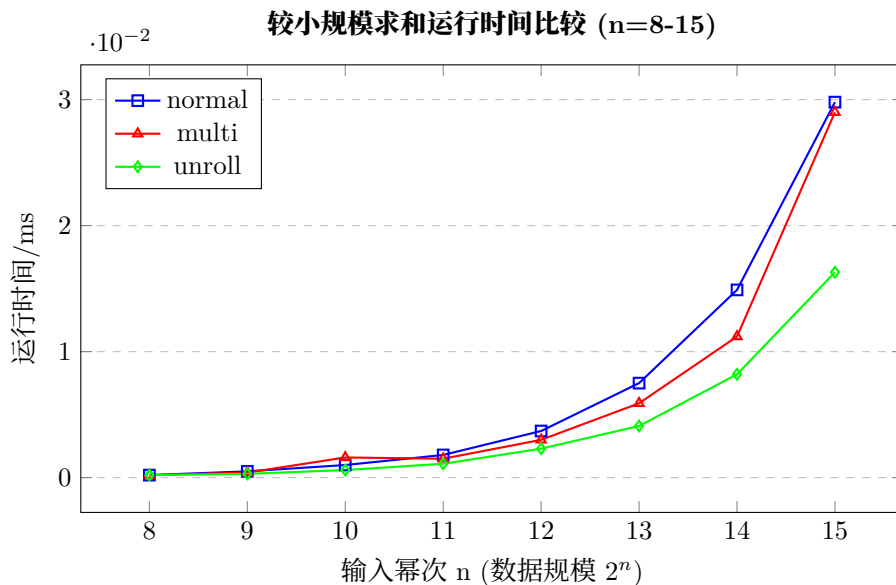


图 4: 较小规模下平凡与多路展开算法运行时间比较

问题规模较小时, 三种算法的运行时间均极短, 折线图中三条曲线几乎重合或交错, 没有拉开明显差距。

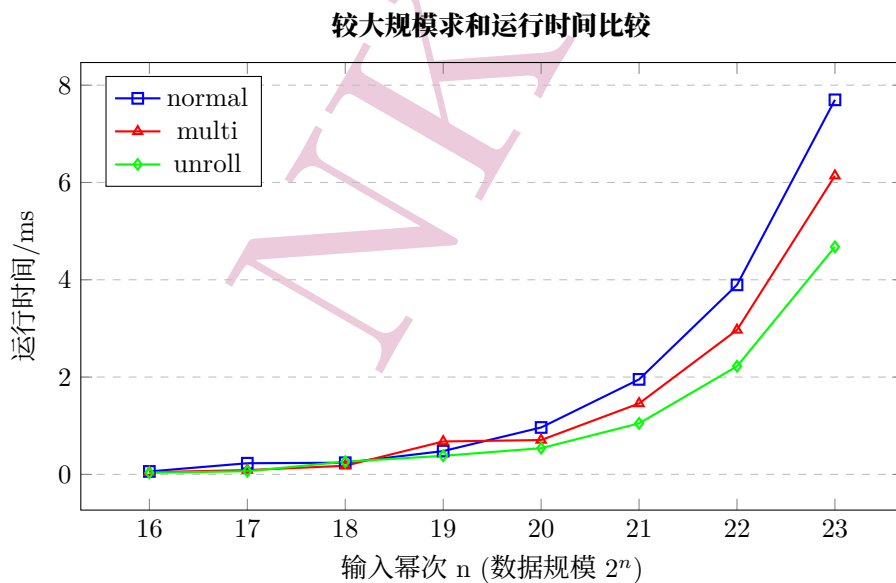


图 5: 较大规模下平凡与多路展开算法运行时间比较

当问题规模逐渐变大后, 由于数据量呈指数级激增, 三条曲线开始出现显著的分野, 且性能差距不断被放大。

可以初步看出采用多路循环展开算法能有效打破传统的链式数据依赖。通过充分利用现代 CPU 的超标量架构来实现指令级并行, 优化算法显著缩短了程序的运行时间; 且随着数据规模的增大与展开路数的增加, 这种底层硬件加速的效益越发显著。

三、 反思总结

(一) 收获

我学习到行连续访问模式对系统执行效率的显著作用，尤其在处理大规模数据集时，优化算法展现出卓越的性能增益。经分析可知，优化代码的高效性根植于对缓存行机制的精准适配，而平凡算法由于缺乏对该存储结构的有效利用，导致访存开销大幅增加。此外，借助两路并行累加策略，有效地缩短了数据依赖链，从而在实践中量化验证了超标量架构强大的指令级并行能力。”

(二) 遇到的问题

当数据规模小时，需要对数据进行处理；输入数据不宜过大，因为数组开辟的大小固定。

四、 代码链接

本项目源代码已上传至 GitHub,可通过以下链接访问:<https://github.com/Yanguozhuang/parallel>